

Programming Principles II

Week 02.



COMT

SOFTWARE CLUSTER

// COMT • **COMPUTATIONAL THINKING** (CIT1C18)



Overview

- Learn how to use operators in Java;
- Learn how to write expressions in Java.



Objectives

- Demonstrate how to use operators in Java;
- Demonstrate how to write expressions in Java.



OPERATORS & EXPRESSIONS



Operators

- Java provides a rich set of operators to manipulate variables. We can divide all the Java operators into the following groups –
 - Arithmetic Operators
 - Relational Operators
 - Bitwise Operators
 - Logical Operators
 - Assignment Operators
 - Misc Operators (Conditional Operator, instanceof Operator, etc..)
- The next few slides will introduce you to the commonly used operators.

Arithmetic Operators

- Arithmetic operators are used in **mathematical expressions** in the same way that they are used in algebra. The following table lists the arithmetic operators (**assuming variable A = 10 and variable B = 20**):

Operator	Description	Example
+ (Addition)	Adds values on either side of the operator.	$A + B = 30$
- (Subtraction)	Subtracts right-hand operand from left-hand operand.	$A - B = -10$
* (Multiplication)	Multiplies values on either side of the operator.	$A * B = 200$
/ (Division)	Divides left-hand operand by right-hand operand.	$B / A = 2$
% (Modulus)	Divides left-hand operand by right-hand operand and returns remainder.	$B \% A = 0$
++ (Increment)	Increases the value of operand by 1.	$B++ = 21$ (same as $B = B + 1$)
-- (Decrement)	Decreases the value of operand by 1.	$B-- = 19$ (same as $B = B - 1$)

Arithmetic Operators: Examples

- Create a Java application call `TestArithmetics.java` as shown on the right.
- Add the codes as shown and run the application to see the results.
- Try to explore by changing the value of the variables and see what happens.

```
1 public class TestArithmetics {  
2  
3     public static void main(String args[]) {  
4         int a = 10;  
5         int b = 20;  
6         int c = 25;  
7         int d = 25;  
8  
9         System.out.println("a + b = " + (a + b) );  
10        System.out.println("a - b = " + (a - b) );  
11        System.out.println("a * b = " + (a * b) );  
12        System.out.println("b / a = " + (b / a) );  
13        System.out.println("b % a = " + (b % a) );  
14        System.out.println("c % a = " + (c % a) );  
15        System.out.println("a++ = " + (a++) );  
16        System.out.println("b-- = " + (b--) );  
17  
18        // Check the difference in d++ and ++d  
19        System.out.println("d++ = " + (d++) );  
20        System.out.println("++d = " + (++d) );  
21    }  
22 }
```



Relational Operators

- The following relational operators are supported in Java. The following table lists the arithmetic operators (**assuming variable A = 10 and variable B = 20**):

Operator	Description	Example
== (equal to)	Checks if the values of two operands are equal or not, if yes then condition becomes true.	(A == B) is not true.
!= (not equal to)	Checks if the values of two operands are equal or not, if values are not equal then condition becomes true.	(A != B) is true.
> (greater than)	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	(A > B) is not true.
< (less than)	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.	(A < B) is true.
>= (greater than or equal to)	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.	(A >= B) is not true.
<= (less than or equal to)	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.	(A <= B) is true.

Relational Operators: Examples

- Create a Java application call `TestRelational.java` as shown on the right.
- Add the codes as shown and run the application to see the results.
- Try to explore by changing the value of the variables and see what happens.

```
1 public class TestRelational {  
2  
3     public static void main(String args[]) {  
4         int a = 10;  
5         int b = 20;  
6  
7         System.out.println("a == b = " + (a == b) );  
8         System.out.println("a != b = " + (a != b) );  
9         System.out.println("a > b = " + (a > b) );  
10        System.out.println("a < b = " + (a < b) );  
11        System.out.println("b >= a = " + (b >= a) );  
12        System.out.println("b <= a = " + (b <= a) );  
13    }  
14 }
```



Logical Operators

- In Java, we use `boolean` data type with logical operators.
- The following table lists the logical operators (assuming `variable A = true` and `variable B = false`):

Operator	Description	Example
<code>&&</code> (logical and)	Called Logical AND operator. If both the operands are non-zero, then the condition becomes true.	<code>(A && B)</code> is false
<code> </code> (logical or)	Called Logical OR Operator. If any of the two operands are non-zero, then the condition becomes true.	<code>(A B)</code> is true
<code>!</code> (logical not)	Called Logical NOT Operator. Use to reverses the logical state of its operand. If a condition is true then Logical NOT operator will make false.	<code>!(A && B)</code> is true

Logical Operators: Examples

- Create a Java application call `TestLogical.java` as shown on the right.
- Add the codes as shown and run the application to see the results.
- Try to explore by changing the value of the variables and see what happens.

```
1 public class TestLogical {  
2  
3     public static void main(String args[]) {  
4         boolean a = true;  
5         boolean b = false;  
6  
7         System.out.println("a && b = " + (a && b));  
8         System.out.println("a || b = " + (a || b));  
9         System.out.println("!(a && b) = " + !(a && b));  
10    }  
11 }
```

Assignment Operators

- Assignment operators are used to assign value to variable.

Operator	Description	Example
=	Simple assignment operator. Assigns values from right side operands to left side operand.	$C = A + B$ will assign value of $A + B$ into C
+=	Add AND assignment operator. It adds right operand to the left operand and assign the result to left operand.	$C += A$ is equivalent to $C = C + A$
-=	Subtract AND assignment operator. It subtracts right operand from the left operand and assign the result to left operand.	$C -= A$ is equivalent to $C = C - A$
*=	Multiply AND assignment operator. It multiplies right operand with the left operand and assign the result to left operand.	$C *= A$ is equivalent to $C = C * A$
/=	Divide AND assignment operator. It divides left operand with the right operand and assign the result to left operand.	$C /= A$ is equivalent to $C = C / A$
%=	Modulus AND assignment operator. It takes modulus using two operands and assign the result to left operand.	$C \% = A$ is equivalent to $C = C \% A$

Assignment Operators: Examples

- Create a Java application call `TestAssignments.java` as shown on the right.
- Add the codes as shown and run the application to see the results. (Hint: You can copy and paste this code)
- Try to explore by changing the value of the variables and see what happens.

```
public class TestAssignments
{
    public static void main(String args[])
    {
        int a = 10;
        int b = 20;
        int c = 0;

        c = a + b;
        System.out.println("c = a + b = " + c );

        c += a ;
        System.out.println("c += a  = " + c );

        c -= a ;
        System.out.println("c -= a = " + c );

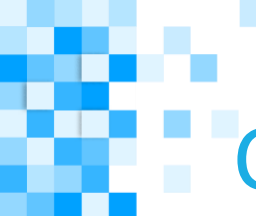
        c *= a ;
        System.out.println("c *= a = " + c );

        a = 10;
        c = 15;
        c /= a ;
        System.out.println("c /= a = " + c );

        a = 10;
        c = 15;
        c %= a ;
        System.out.println("c %= a  = " + c );
    }
}
```



COMMENTS



Comments in Java

- In previous code examples, you would have seen certain lines that starts with “`//`”.
- The double forward slashes are for adding comments.
- Java supports single-line and multi-line comments very similar to C and C++.
- All characters available inside any comment are ignored by Java compiler.
- Comments allow you to insert notes or description at certain lines of the codes.
- A comment will not be executed as an instruction.

```
public class MyFirstJavaProgram {  
  
    /* This is my first java program.  
     * This will print 'Hello World' as the output  
     * This is an example of multi-line comments.  
     */  
  
    public static void main(String []args) {  
        // This is an example of single line comment  
        /* This is also an example of single line comment. */  
        System.out.println("Hello World");  
    }  
}
```



SELECTIONS

What are Selections?

- Selections are statements that provide a way for an application to determine decisions.
- For example, in a grading scenario, you may write some codes for an application to determine a student's grade and output a message.
- These codes can be in the form of `if..else` statement.
- Or the `switch` statement.

Conditions must be in brackets.

```
char grade;  
  
if (grade == 'A')  
{  
    System.out.println("Well done!");  
}  
else  
{  
    System.out.println("Try again!");  
}
```



Formats

- The format of a simple **if** selection or decision statement is in the form as described here.
- Every condition is surrounded by brackets.

```
if (<condition>
{
    <instructions>
}
```

```
public class Test
{
    public static void main(String args[])
    {
        int x = 10;

        if ( x < 20 )
        {
            System.out.print("This is if statement");
        }
    }
}
```



Formats

- There can be an **else** if there is an otherwise condition.

```
if (<condition>
{
    <instructions>
}
else
{
    <instructions>
}
```

```
public class Test
{
    public static void main(String args[])
    {
        int x = 30;

        if ( x < 20 )
        {
            System.out.print("This is if statement");
        }
        else
        {
            System.out.print("This is else statement");
        }
    }
}
```

- There can be multiple conditions where we can use **Logical** operators to form.

```
if ((<condition1>) && (<condition2>))  
{  
    <instructions>  
}  
else  
{  
    <instructions>  
}
```

```
public class Test  
{  
    public static void main(String args[])  
    {  
        int x = 30;  
  
        if (( x > 10 ) && ( x <= 20 ))  
        {  
            System.out.print("This is if statement");  
        }  
        else  
        {  
            System.out.print("This is else statement");  
        }  
    }  
}
```



Formats

- There can be multiple **if** conditions.

```
if (<condition1> && (<condition2>))
{
    <instructions>
}
else if (<condition3>)
{
    <instructions>
}
```

```
public class Test
{
    public static void main(String args[])
    {
        int x = 30;

        if ( x == 10 )
        {
            System.out.print("Value of X is 10");
        }
        else if ( x == 20 )
        {
            System.out.print("Value of X is 20");
        }
        else if ( x == 30 )
        {
            System.out.print("Value of X is 30");
        }
        else
        {
            System.out.print("This is else statement");
        }
    }
}
```



Formats

- There can be nested **if** conditions.

```
public class Test
{
    public static void main(String args[])
    {
        int x = 30;
        int y = 10;

        if ( x == 30 )
        {
            if ( y == 10 )
            {
                System.out.print("X = 30 and Y = 10");
            }
        }
    }
}
```

```
if (<condition1> && (<condition2>))
{
    <instructions>
    if (<condition3>)
    {
        <instructions>
    }
    <instructions>
}
else
{
    <instructions>
    if (<condition4>)
    {
        <instructions>
    }
    <instructions>
}
```

Example

- Let's see an example of a login scenario.
- Here you will learn how to read input from the scanner and use the decision statement to determine the input.
- Create a Java app called `TestLogin.java` and add the codes on the right.
- In line 13, we use a built-in function called “`equals`” to compare strings.
- Run the application and enter “`mypass`” at the prompt.
- In the next run, enter any text to see what happens.

```
import java.util.Scanner;

public class TestLogin {
    public static void main(String[] args){
        boolean login = false;
        String password = "mypass";
        System.out.println("Enter password: ");
        Scanner scanner = new Scanner(System.in);
        String input = scanner.nextLine();

        if(input.equals(password)){
            login = true;
            System.out.println("Login success = " + login);
        }
        else{
            System.out.println("Login success = " + login);
        }
    }
}
```



Fun Quiz

- What is wrong with the following code fragment? Will this code compile if it is part of an otherwise valid program?

```
if (length = MIN_LENGTH)
    System.out.println("The length is minimal.");
```




Fun Quiz

- What output is produced by the following code fragment?

```
int num = 87;  
int max = 25;  
  
if (num >= max * 2)  
    System.out.println("apple");  
    System.out.println("orange");  
System.out.println("pear");
```

- What output is produced by the following code fragment?

```
int limit = 100;
int num1 = 15;
int num2 = 40;

if (limit <= limit)
{
    if (num1 == num2)
        System.out.println("lemon");
    System.out.println("lime");
}
System.out.println("grape");
```



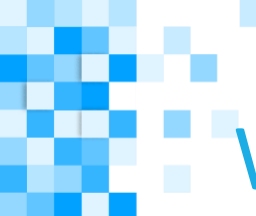
Challenge

1. In the same `TestLogin` application, output “Welcome to the jungle!” if the password is correct, else output “You are not authorised!” if the password is incorrect.
2. Create a new application called `TestMarks`. Use the table below to write decision codes to output the corresponding statement based on the conditions. (Refer to the slides on Relational and Logical operators for reference.)

Conditions for Mark	Output
Less than 50	“You have failed.”
Between 50 and 59 inclusive	“You score a ‘D’.”
Between 60 and 69 inclusive	“You score a ‘C’.”
Between 70 and 79 inclusive	“You score a ‘B’.”
Above 80	“You score an ‘A’.”



LOOP



What is a FOR Loop?

- Suppose you want to execute a block of instructions multiple times, how do you tell the computer to do that?
- We can use a loop to instruct the computer how to repeat.
- Java supports the following loops:
 - for loop
 - while loop
 - do..while loop
- For example, if we want to count from 1 to 5, we can write the code using **for** loop as shown on the right.

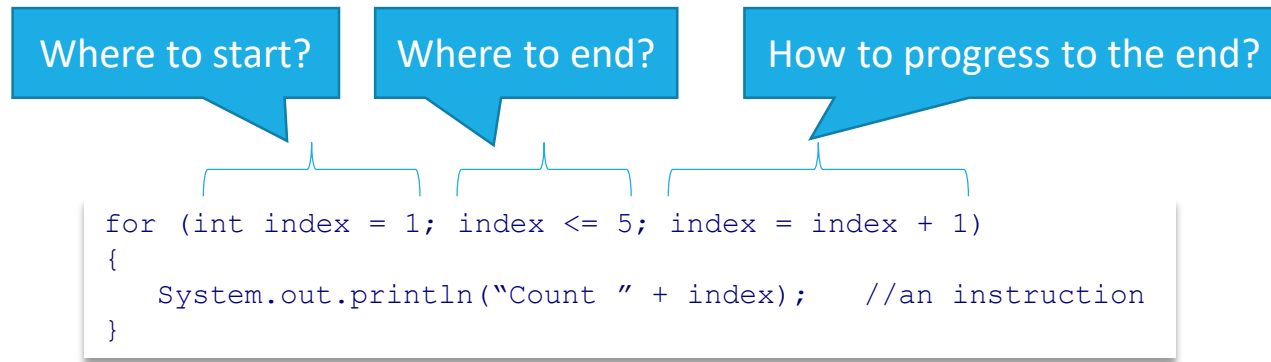
```
public class TestSwitch
{
    public static void main(String args[])
    {
        for (int index = 1; index <= 5; index = index + 1)
        {
            System.out.println("Count " + index);
        }
    }
}
```

Output:

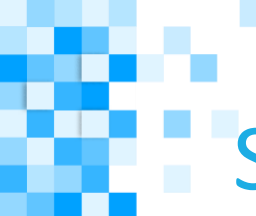
```
Count 1
Count 2
Count 3
Count 4
Count 5
```

What is a FOR Loop?

- Let's take a closer look at the `for` loop.



- In the above codes, the `for` loop is divided into 3 parts.
- The first part tells the computer where to start; the second part tells the computer where to end; the third part tells the computer how to progress to the end from the start.
- The `index` variable, which is an integer, acts as a **counter**. It starts with the value of 1. The third part is a Math expression that tells the computer to increase the `index` by 1 after every loop.
- A loop is completed when the instructions in the `for` loop completes. There can be multiple lines of instructions within a `for` loop.
- Hence, in the above example, the loop will end when `index` reaches 5.



Structure of a FOR Loop

- The **for** loop repeats a set of instructions a **fixed number of times**.
- The structure of a **for** loop is shown in the example below.

```
<instruction 1>
<instruction 2>
for (<where to start?>; <where to end?>; <how to progress to the end?>)
{
    <instruction 3>
    <instruction 4>
}
<instruction 5>
<instruction 6>
```

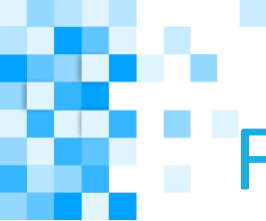
- Once a **for** loop ends, the instruction 5 will be executed followed by instruction 6.



Fun Quiz

- How can you re-write the third part: (index = index + 1)? (Hint: Look at the slide for Arithmetic Operators)

```
for (int index = 1; index <= 5; index = index + 1)
{
    System.out.println("Count " + index);    //an instruction
}
```

Fun Quiz

- What output is produced by the following code fragment?

```
for (int num = 0; num <= 200; num += 2)
    System.out.println(num);
```



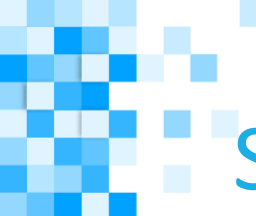
Challenge

1. Write a `for` loop to print the odd numbers from 1 to 99 (inclusive).
2. Write a `for` loop to print the multiples of 3 from 300 down to 3.



Summary

- In this lesson, you have learned the usage of various operators, selection statements and loop.
- In the next lesson, you will learn how to group instructions and reuse them.



Supplementary Materials

Topic	Source
Java Operators	https://www.tutorialspoint.com/java/java_basic_operators.html
Java Selections	https://www.tutorialspoint.com/java/java_decision_making.htm
Java Loop	https://www.tutorialspoint.com/java/java_loop_control.htm

THE END



COMT